

Lab 2: Gradient Descent and Newton

Step sizes, SGD versus batch, and a badly conditioned quadratic.

Work in a **Jupyter** notebook. Number the exercises. Do notes **2.1** through **2.3** first. You will need `numpy` and `matplotlib`.

The original assignment PDF is [lab-2-assignment.pdf](#). Class provided a snippet that builds linear-looking data; the generator below is enough.

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
m = 100
X = 2 * np.random.rand(m, 1)
y = 4 + 3 * X + np.random.randn(m, 1)
X_b = np.c_[np.ones((m, 1)), X] # intercept column
```

1. Three gradient methods, in words (1 point)

In a short markdown cell: how do **batch** GD, **mini-batch** GD, and **SGD** differ (gradient, noise, cost per step)? No code.

2. Paths on linear data (2 points)

MSE for $\theta = (b, w)$ on the data above.

1. Batch GD, 1000 iterations, **fixed** rates (0.02), (0.1), (0.5). Plot the first 10 steps in the (b, w) plane for each rate. Which rate converges fastest without blowing up?
2. SGD with a decaying schedule and a random start. Plot the first 20 steps. What do you see?
3. Mini-batch GD, batch size **20**. Plot the first 20 steps.
4. Overlay the three paths. Which one is smoothest? Which one makes progress fastest at the beginning?

3. Newton versus GD on a bad quadratic (2 points)

[$J(\theta) = \frac{1}{2} \theta^T H \theta + \frac{\lambda}{2} \|\theta\|^2$, $\lambda = 10$.]

Start from a random $(\boldsymbol{\theta} \in \mathbb{R}^2)$. Run Newton (the Hessian of this J is $(H + \lambda I)$, constant). Then run gradient descent. Compare iteration counts to a small tolerance. A walkthrough of Newton is [here](#).

The condition number is the point of the exercise. Say so in a comment.